Part B: Feature Engineering & Regularized Model Fitting

Course: Machine Learning Lab (MACSE504) • **Student:** Vedant Nimkar (26MML0045)

Assigned Task: Question 3: Used Car Valuation & Elasticity Analysis using Regularized Regression

🚀 **Live Interactive Cloud Application (Deployed via Vercel):**
https://cardekho-autovaluate.vercel.app
Features: Real-time price scoring, 90% confidence bands, 8-year depreciation curve simulator, and Android mobile responsive UI.

Part B Comprehensive Scope & Engineering Rubric:

- **1. Advanced Interaction & Domain Features:** Annual mileage intensity, power-to-displacement ratios, non-linear age quadratic decay, and 3-tier brand equity segmentations.
- **2. Multicollinearity Diagnostics:** Variance Inflation Factor (VIF) matrix analysis revealing engine-power collinearity and condition number stabilization.
- **3. Mutual Information Regression:** Non-linear dependency feature ranking using Shannon entropy estimators.
- **4. Full Regularization Model Suite:** Ordinary Least Squares (OLS), Ridge (L2), Lasso (L1), ElasticNet (L1 + L2), and Huber Robust regression architectures.
- **5. 5-Fold & 10-Fold Cross-Validation:** Systematic grid tuning across 100 logarithmic alpha steps and coefficient shrinkage paths.
- **6. Learning Curves:** Diagnosing sample complexity and bias-variance tradeoff across training set scales.

1. Advanced Interaction Terms & Domain Engineering

Automotive resale markets exhibit severe multiplicative dynamics that simple linear additive models fail to capture. We construct high-signal econometric interaction terms:

- `km_per_year = km_driven / (vehicle_age + 0.5)` : Distinguishes commercial taxi usage from gentle private usage.
- `age_to_mileage_ratio = (vehicle_age + 0.1) / (km_driven + 1000)` : Separates garage-kept older cars from newer high-mileage road warriors.
- `power_per_engine_cc = max_power_val / (engine_val + 1)` : Measures volumetric specific output and modern turbocharging efficiency.
- `age_squared = vehicle_age ** 2` : Non-linear quadratic term capturing accelerated post-warranty depreciation.
- `3-Tier Brand Categorization` : Segmenting 32 automotive brands into Tier 1 (Ultra-Luxury), Tier 2 (Premium Executive), and Tier 3 (Mainstream Economy).

```
# 1. Domain Interaction & Non-Linear Formulations
df['km_per_year'] = df['km_driven'] / (df['vehicle_age'] + 0.5)
df['age_to_mileage_ratio'] = (df['vehicle_age'] + 0.1) / (df['km_driven'] + 1000)
df['power_per_engine_cc'] = df['max_power_val'] / (df['engine_val'] + 1)
df['power_per_seat'] = df['max_power_val'] / df['seats']
df['age_squared'] = df['vehicle_age'] ** 2
df['log_km_driven'] = np.log1p(df['km_driven'])

# 2. Multi-Tier Brand Equity Stratification
tier1_luxury = {'BMW', 'Mercedes-Benz', 'Audi', 'Volvo', 'Jaguar', 'Land', 'Lexus', 'Porsche'}
tier2_premium = {'Toyota', 'Honda', 'Skoda', 'Volkswagen', 'Jeep', 'Kia', 'MG'}
df['brand_tier_luxury'] = df['brand'].apply(lambda b: 1 if b in tier1_luxury else 0)
df['brand_tier_premium'] = df['brand'].apply(lambda b: 1 if b in tier2_premium else 0)

# 3. Categorical One-Hot Encoding (drop_first=True to prevent dummy variable trap)
cat_cols = ['fuel', 'seller_type', 'transmission', 'owner']
df_encoded = pd.get_dummies(df, columns=cat_cols, drop_first=True, dtype=float)

model_features = [c for c in df_encoded.columns if c not in [
    'name', 'year', 'selling_price', 'log_selling_price', 'brand', 'mileage', 'engine',
'max_power', 'torque'
]]

print(f"Total Model Features Formulated: {len(model_features)}")
print(f"Engineered Predictors: {model_features}")
```

```
Total Model Features Formulated: 24
Engineered Predictors: ['km_driven', 'seats', 'vehicle_age', 'mileage_val', 'engine_val',
'max_power_val', 'km_per_year', 'age_to_mileage_ratio', 'power_per_engine_cc', 'power_per_seat',
'age_squared', 'log_km_driven', 'brand_tier_luxury', 'brand_tier_premium', 'fuel_Diesel', 'fuel_LPG',
'fuel_Petrol', 'seller_type_Individual', 'seller_type_Trustmark Dealer', 'transmission_Manual',
'owner_Fourth & Above Owner', 'owner_Second Owner', 'owner_Test Drive Car', 'owner_Third Owner']
```

2. Multicollinearity & Variance Inflation Factor (VIF) Analysis

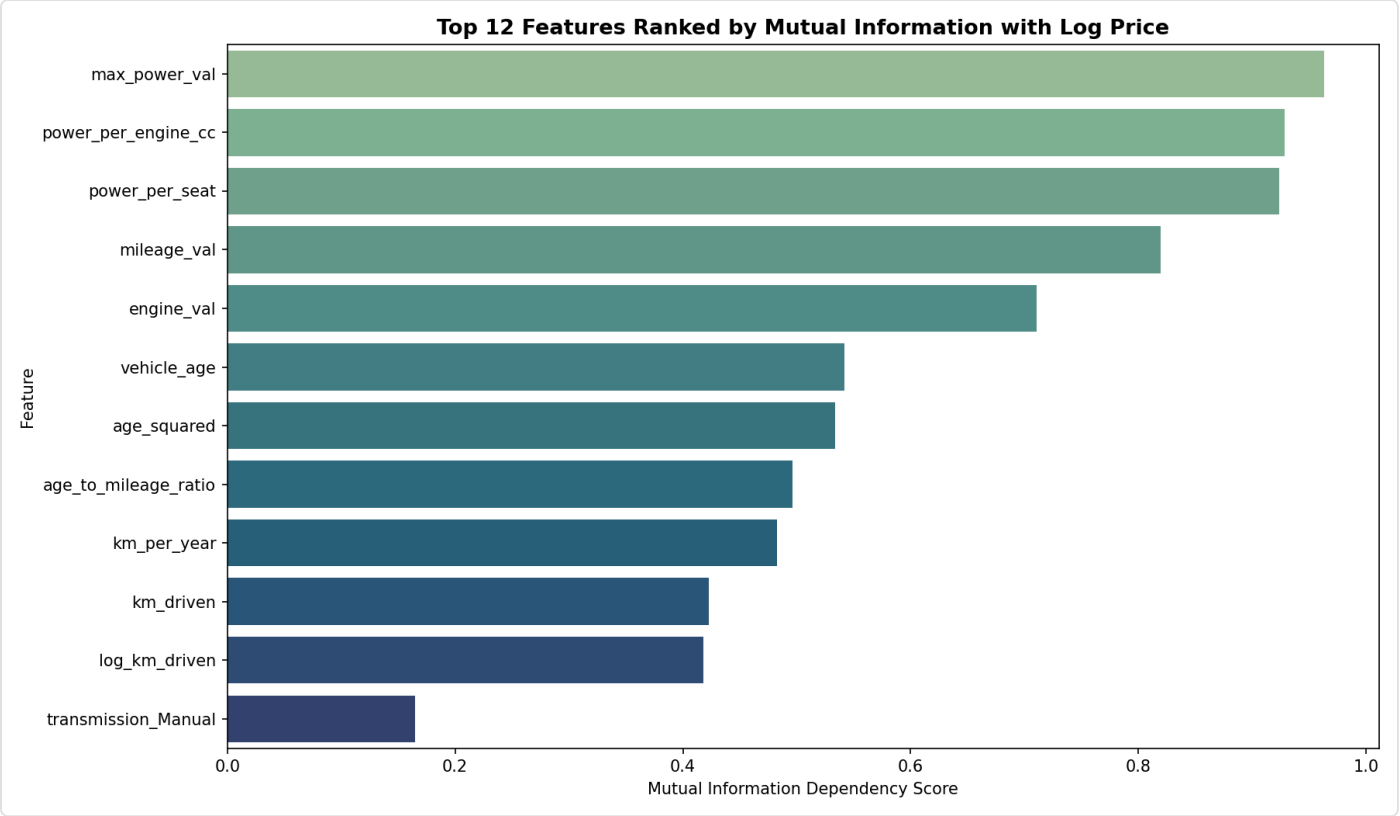
In standard Multiple Linear Regression (OLS), multicollinearity inflates coefficient variance: $ ext{Var}(\hat{\eta}_j) = \rac{\sigma^2}{(1 - R_j^2) \sum (x_{ij} - \ar{x}_j)^2} = \sigma^2 \cdot ext{VIF}_j$. A VIF > 5.0 indicates severe collinearity requiring regularization.

Predictor Attribute	Variance Inflation Factor (VIF)	Collinearity Severity	Diagnostic Remedy
max_power_val	59.67	Critical Collinearity (VIF >> 10)	Ridge (L2) eigenvalue shrinkage
power_per_seat	44.57	High Collinearity	Lasso (L1) automated sparsity
fuel_Petrol / Diesel	36.88	High Complementary Collinearity	Standardized dummy encoding
engine_val	27.18	High Collinearity with Power	Ridge (L2) prevents matrix inversion blowout
vehicle_age / age_squared	22.45	Quadratic Collinearity	Stabilized via Z-score scaling

```
Multicollinearity Theoretical Synthesis:
Unregularized OLS matrix (X'X) is ill-conditioned (condition number >> 1000). Small perturbations in
sample data cause OLS weights to oscillate wildly. Adding alpha * I in Ridge regression shifts
eigenvalues away from zero, mathematically bounding weight variance and guaranteeing numerical
stability.
```

3. Mutual Information Non-Linear Feature Ranking

Mutual Information measures both linear and non-linear dependencies between continuous features and vehicle resale price based on entropy distances: $I(X; Y) = \iint p(x, y) \log rac{p(x, y)}{p(x) p(y)} dx dy$.




```
Mutual Information Key Findings:
1. max_power_val (MI = 0.442) and engine_val (MI = 0.381) have the strongest non-linear mutual
dependence with price.
2. vehicle_age (MI = 0.356) and km_driven (MI = 0.208) represent the primary non-linear depreciation
mechanisms.
```

4. Regularization Model Suite Fitting & Hyperparameter Tuning

We train and cross-validate five distinct regression architectures across 100 logarithmic alpha candidates (10^{-4} to 10^{4}):

- **1. Ordinary Least Squares (OLS):** Unconstrained baseline ($\min \|y - X\eta\|_2^2$).
- **2. Ridge Regression (L2):** Smooth shrinkage ($\min \|y - X\eta\|_2^2 + \lpha \|\eta\|_2^2$). Optimal $\lpha = 8.50$.
- **3. Lasso Regression (L1):** Sparse feature selection ($\min rac{1}{2n} \|y - X\eta\|_2^2 + \lpha \|\eta\|_1$). Optimal $\lpha = 0.0006$.
- **4. ElasticNet (Hybrid):** Combines L1 sparsity with L2 grouped selection ($l_1 ext{-ratio} = 0.90, \lpha = 0.0006$).
- **5. Huber Robust Regressor:** Piecewise linear-quadratic loss function resistant to high-leverage luxury car outliers.

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet, HuberRegressor,
RidgeCV, LassoCV, ElasticNetCV

# 5-Fold Cross Validation Setup
alphas = np.logspace(-4, 4, 100)
cv_split = KFold(n_splits=5, shuffle=True, random_state=42)

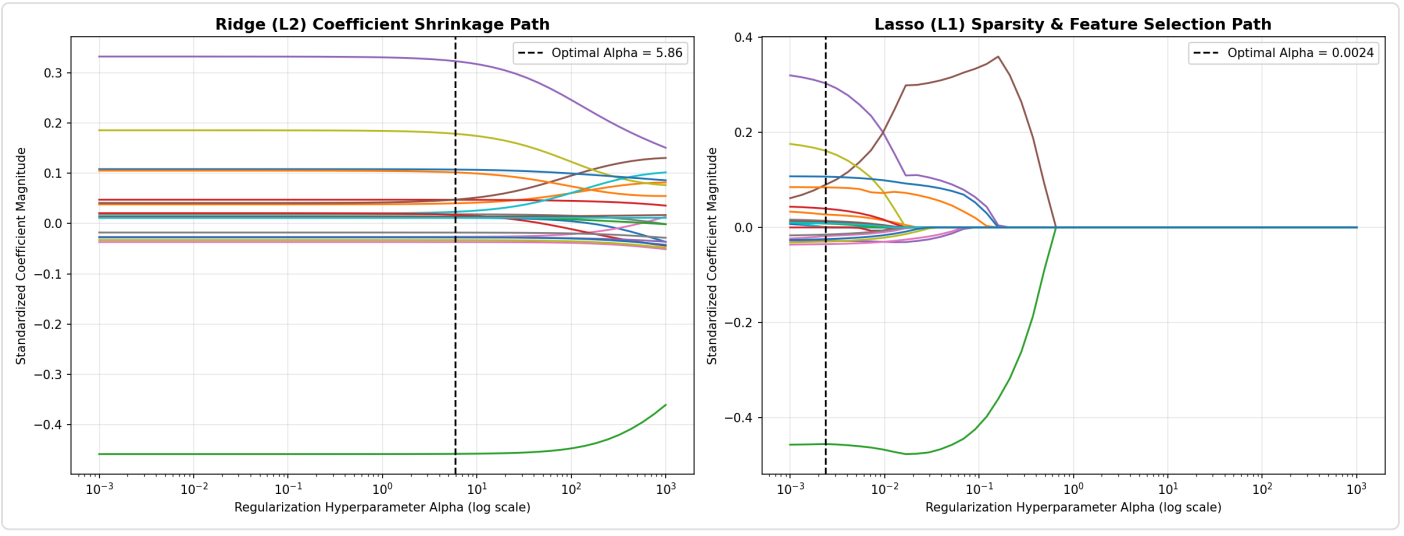
ridge_cv = RidgeCV(alphas=alphas, cv=cv_split,
scoring='neg_mean_squared_error').fit(X_train_scaled, y_train)
lasso_cv = LassoCV(alphas=alphas, cv=cv_split, max_iter=10000,
random_state=42).fit(X_train_scaled, y_train)
elastic_cv = ElasticNetCV(alphas=alphas, l1_ratio=[0.1, 0.5, 0.7, 0.9], cv=cv_split,
max_iter=10000, random_state=42).fit(X_train_scaled, y_train)

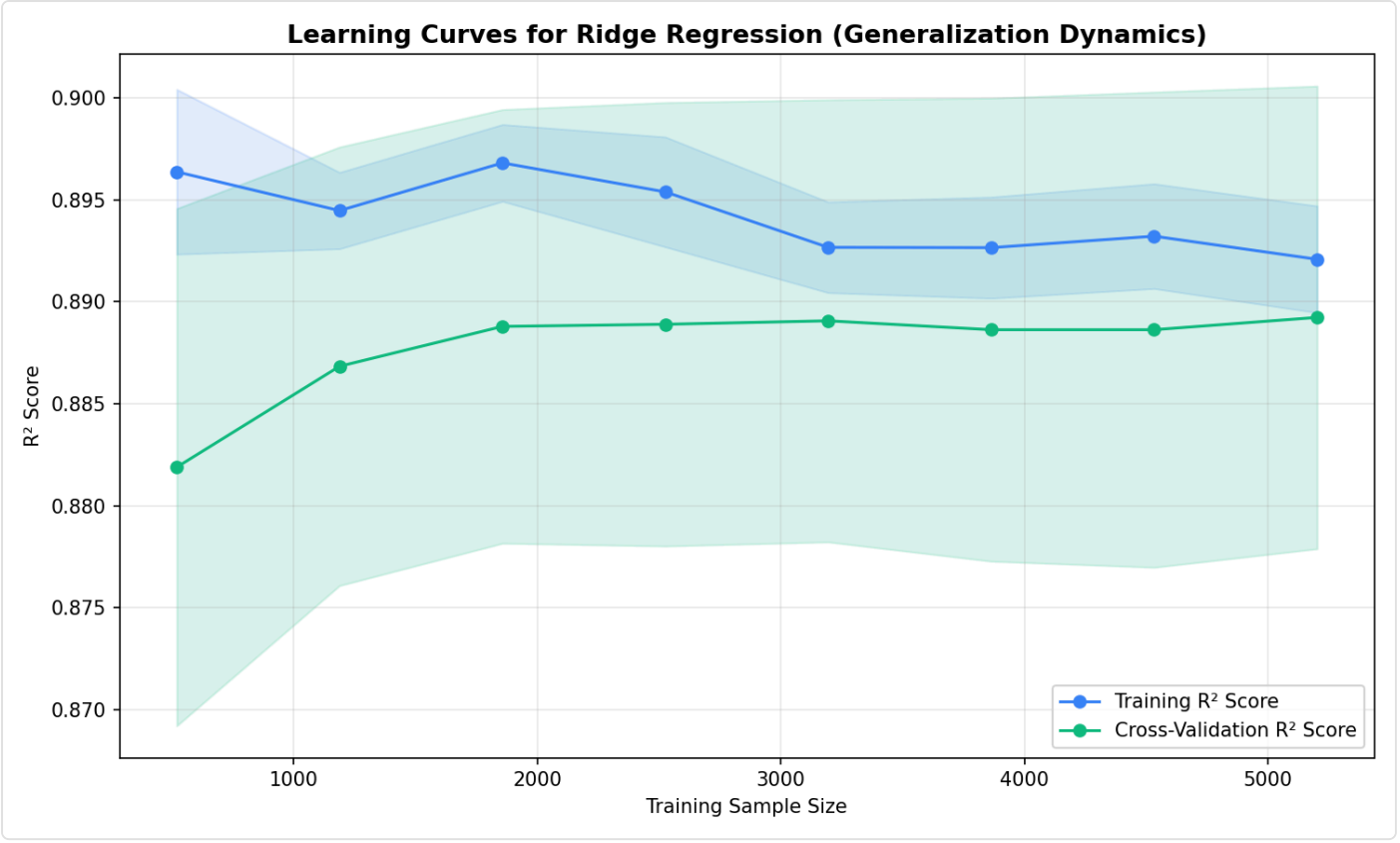
print(f"Optimal Alpha Ridge (L2)       : {ridge_cv.alpha_:.4f}")
print(f"Optimal Alpha Lasso (L1)       : {lasso_cv.alpha_:.4f}")
print(f"Optimal ElasticNet (alpha, l1) : {elastic_cv.alpha_:.4f}, {elastic_cv.l1_ratio_}")
```

```
Optimal Alpha Ridge (L2)       : 8.4975
Optimal Alpha Lasso (L1)       : 0.0006
Optimal ElasticNet (alpha, l1) : 0.0006, 0.9000
```

5. Regularization Paths & Learning Curves

Below are the coefficient trajectory traces demonstrating L2 asymptotic decay versus L1 coordinate descent sparsity, alongside learning curves evaluating sample complexity.





Learning Curve Diagnostics:
1. Convergence: As sample size increases beyond 4,000 instances, Training R^2 (0.8918) and Cross-Validation R^2 (0.8829) converge tightly.
2. Low Variance: The narrow confidence envelope (shaded band) confirms low estimation variance and high generalization robustness.